📘 Object Oriented Systems (OOS) — 2021 Question 2 Complete Detailed Solutions

🚀 Question 2(a): Static Import & Sub-packages

📘 Static Import

Static import allows direct access to static methods and variables of a class without specifying the class name [cite: source]. Introduced in Java 5, it improves code readability when specific utility members are used frequently [cite: source].

Syntax:

```
import static package.ClassName.member;
// OR
import static package.ClassName.*;
```

Utility Example: Using static import for the `Math` class avoids repetitive prefixes like `Math.sqrt()` [cite: source].

```
import static java.lang.Math.*;

class Demo {
    public static void main(String args[]) {
        System.out.println(sqrt(25)); // 5.0
        System.out.println(pow(2,3)); // 8.0
        System.out.println(PI);      // 3.14...
    }
}
```

🚀 Creating a Sub-package

Sub-packages provide hierarchical organization for classes, mapping directly to directory structures [cite: source].

Steps to create `college.student`:

1. **Define Package:** Use `package college.student;` at the top of the file [cite: source].
2. **Directory Structure:** Store the file in a folder path matching the name (`college/student/`) [cite: source].
3. **Compile:** Use `javac -d . FileName.java` to generate the structure [cite: source].

🚀 Question 2(b): Multilevel Inheritance

In multilevel inheritance, a hierarchy is formed where a class inherits from a parent, which in turn inherits from another parent (`Base -> Child -> GrandChild`) [cite: source].

Key Requirements:

- **Constructor Chaining:** Parent constructors must execute before child constructors [cite: source].
- **Method Overriding:** A child class provides a specific implementation of a method already defined in its superclass [cite: source].
- **Super Keyword:** Used to call the immediate parent's constructor or methods [cite: source].

Java Program Scenario:

```
class Base {
    Base(int x) { System.out.println("Base Constructor: " + x); }
    void fun() { System.out.println("Base fun()"); }
}

class Child extends Base {
    Child(int x, int y) {
        super(x); // Invokes Base constructor
        System.out.println("Child Constructor: " + y);
    }
    @Override
    void fun() { System.out.println("Child fun()"); }
}

class GrandChild extends Child {
    GrandChild(int x, int y) {
        super(x, y); // Invokes Child constructor
        System.out.println("GrandChild Constructor");
    }
    void showBaseFun() {
        super.fun(); // Calls Child's fun() (immediate parent)
    }
}
```

🚀 Question 2(c): Generic Binary Search

Generics enable type-safe, reusable algorithms that work across multiple data types [cite: source]. For binary search, the type must implement the `Comparable` interface to allow element comparison [cite: source].

📊 Logic Overview

Binary search achieves **O(log n)** complexity by repeatedly dividing the sorted search space in half [cite: source].

Generic Implementation:

```java
class Search<T extends Comparable<T>> {
    void find(T arr[], T key) {
        int low = 0, high = arr.length - 1;
        boolean found = false;
        while(low <= high) {
            int mid = (low + high) / 2;
            int result = key.compareTo(arr[mid]);
            if(result == 0) {
                System.out.println("Found at index: " + mid);
                found = true; break;
            } else if(result > 0) low = mid + 1;
            else high = mid - 1;
        }
        if(!found) System.out.println("Not Found");
    }
}
```

🧠 JVM Behavior: Type Erasure

During compilation, the JVM performs **Type Erasure**, replacing generic type parameters with `Object` or the upper bound (`Comparable` in this case) [cite: source]. This ensures backward compatibility with older Java versions [cite: source].

🏁 Final Conclusion

These solutions cover essential OOS concepts including modularity (packages), hierarchy (inheritance), and type-safe reusability (generics), forming the foundation of robust Java application design [cite: source].